

從 Wiki 到 Winning：英文基礎的培養與成果

課程連結

這份報告整理了我在課外透過技術社群與日常使用累積的英語能力，以及如何與「英文閱讀與寫作」的課堂內容相互印證。

學習背景

我的英語能力並非來自刻意的考試準備，而是在技術學習與日常娛樂中自然累積的。從國二開始瀏覽 **Reddit** 論壇起，英文就成為我取得資訊的主要語言。從技術類內容、動漫討論，到 **Twitter** 上的各種社群互動，這種「不是在學英文，而是剛好都在用英文」的狀態，讓語言能力在不知不覺中持續成長。

技術場景中的英語沉浸

在從零安裝 **Arch Linux** 的過程中，我必須逐字閱讀 **Arch Wiki** 的全英文安裝指南或是看英文教學影片。在讀不懂就裝不起來、沒有中文版可以退的壓力下，久了自然就習慣直接讀原文而不是等翻譯。

日常生活中的英語輸入

Reddit 是我國中以來每天都會瀏覽的論壇，從 **r/jailbreak** 的技術討論到動漫、遊戲相關的 **subreddit** 如 **r/BocchiTheRock**，大量英文閱讀就這樣混進了日常生活裡。**Twitter** 上追蹤的藝術家跟政治與國際新聞也是持續的英語輸入來源。這些看起來不算學習的行為，累積起來對詞彙量與語感的影響非常大。

開源協作中學到的跨國溝通

我在 **GitHub** 上大約有 150 個 **pull request** 被合併，涵蓋英翻中翻譯、功能開發與各種 **bug fixes**。這些提交很多都必須用英文與來自不同國家的維護者溝通。

用 **git submodule** 簡化重複流程

在為 **wallpaper-cava** 這個 **Rust** 專案貢獻時，我注意到原本的 **build** 步驟要求使用者手動 **clone** 兩次 **repo**，一次是主專案，一次是依賴的 **wayland-rs**。我提交了一個 **PR** 用 **git**

submodule 取代手動 **clone**，讓使用者只需要 **git clone --recursive** 一行就能完成所有準備。維護者與我在六則留言的對話後合併了這個修改，讓我發現效率的重要性不亞於功能性。

跨國 code review 中的規範學習

向 **nvim-web-devicons** 提交檔案類型圖示支援時，因為範圍太大且自行標記 **review comment** 為已解決，加上未遵守命名規範，被三位 **reviewer** 反覆要求修改。最後維護者在合併時給出了三條建議避免重蹈覆轍。這次經驗讓我徹底理解「**atomic PR**」與「**尊重 reviewer 流程**」的基本紀律。

跨環境相容性的實戰經驗

向 iOS 越獄插件 **flora** 提交修正 **shell** 顏色輸出的 **PR** 時，沒有考慮到 **sh** 與 **bash** 對 **echo -e** 的行為差異，導致修正在 **Debian** 環境下反而產生錯誤。維護者合併後親自補上了相容寫法，我在後續討論中追查了 **sh**、**bash**、**zsh** 三種環境的實際行為差異。這是我第一次意識到「**works on my machine**」帶來的缺點。

從翻譯到架構建議

為系統更新工具 **topgrade** 提交繁體中文翻譯時，我主動建議維護者將 **i18n** 架構從單一 **YAML** 改為 **po/mo** 分檔。建議最終未被採納，但維護者清楚解釋了集中管理在他們專案脈絡下的優勢。所有的技術決策都必須放回它所處的脈絡中才有意義。

回顧

從連 **commit** 都會打錯、被修正協作規範，到能主動提出架構建議，每一次 **PR** 學到的不只是英文，更是跨國協作的重要技能。

成果驗證

TOEIC Listening & Reading 960 分。在沒有額外準備的狀況下拿到這個成績，驗證了長期處在英語環境對聽力與閱讀的實際效果。

課堂與課外的連結

「英文閱讀與寫作」這堂課給了系統性的語言框架，強化了我在學術文本方面的閱讀



策略與結構化寫作能力。課外則提供了大量真實的應用場景跟溝通壓力，兩者互補讓英文實力建立在真正的實用性上。

反思

英語對我來說不是一個要特別「學」的科目，而是每天都在用的工具。從逛 **Reddit** 到寫 PR、從讀 **Arch Wiki** 到解釋 **iOS tweaks**，這些事情本身就是無意識的語言訓練。總結這段經歷，想把語言學好，最快的方法就是把自己丟進沒辦法依賴中文的環境裡。

附錄

代表性 Pull Request 連結

1. **rs-pro0/wallpaper-cava #6** (用 submodule 簡化 build 流程)
<https://github.com/rs-pro0/wallpaper-cava/pull/6>
2. **acquitelol/flora #5** (Shell 顏色輸出修正)
<https://github.com/acquitelol/flora/pull/5>
3. **nvim-tree/nvim-web-devicons #422** (檔案類型圖示支援)
<https://github.com/nvim-tree/nvim-web-devicons/pull/422>
4. **topgrade-rs/topgrade #931** (英翻中) <https://github.com/topgrade-rs/topgrade/pull/931>

開源軟體繁體中文在地化翻譯工具：[replace.sh](#)

課程連結

這份報告是我將自己在課外開發的翻譯工具，結合「專題閱讀與研究」課程要求所整理出的完整專題。

一、研究動機

在長期參與開源社群的過程中，我發現大量專案的繁體中文翻譯是從簡體直接轉換而來，充斥著不符台灣習慣的詞彙（如軟體被寫成軟件、記憶體被寫成內存），甚至有些專案全無繁體中文版本。

起初我手動逐行修正並提交 **pull request**，但同樣的錯誤在不同專案反覆出現，手動處理根本無法應對數十個專案的規模。因此我開發了自動化工具 **replace.sh** 來解決這個重複性的問題。

二、技術架構與功能

核心功能

- 語言：**Bash**（約 1800 行）
- 檔案搜尋：**fd + ripgrep**（高速檔案發現與內容匹配）
- 轉換核心：**OpenCC** 簡繁轉換引擎 + 自訂 **sed** 字典檔輔助（**dict.txt**）
- 支援格式：純文字、**json**、**png(metadata)** 等
- 支援方向：**CN→TW**（預設） / **TW→CN**（反向模式）

額外功能

- 檔名同步翻譯：處理檔案內容的同時自動轉換檔名中的簡體字
- 並行處理模式（**-p**）：面對大量檔案時分散運算提升效率
- **git** 整合：讀取 **.gitignore**（**-g**），自動加入版本控制暫存區（**--stage**）
- 安全機制：備份（**-b**）、**dry-run** 預覽（**-n**）、日誌輸出（**-l**）、覆寫警告（可用 **-no-confirm** 忽略）

歧義詞分層處理

部分詞彙在繁體中文有多重含義，不能無條件替換。例如「質量」同時表示品質與物理學上的質量，若一律轉換會造成科學文本的謬誤。因此我將替換規則分為主字典（dict.txt）與爭議性字典（controversial.txt）兩層，後者需要使用者以 -c 參數主動啟用，避免誤判。

三、開發歷程

初期：單純的字串替換

最早版本只是一支簡單的 sed 替換腳本，處理明確對應的詞彙。不過初期的腳本問題連連。程式碼區塊內的特定英文字串被意外修改，或者部分單字被錯誤轉換都是之後需要解決的。

中期：引入分層替換與白名單

為了解決誤判，我在架構中引入 OpenCC 作為基礎轉換引擎，再用自訂的 sed 字典檔處理 OpenCC 無法涵蓋的臺灣特用詞。同時將歧義詞獨立成 controversial 字典，由使用者自行決定啟用與否。開發過程中我使用 AI 工具輔助實作各功能模組的程式碼，自己則專注在功能規劃、歧義詞字典的維護與輸出品質的把關。

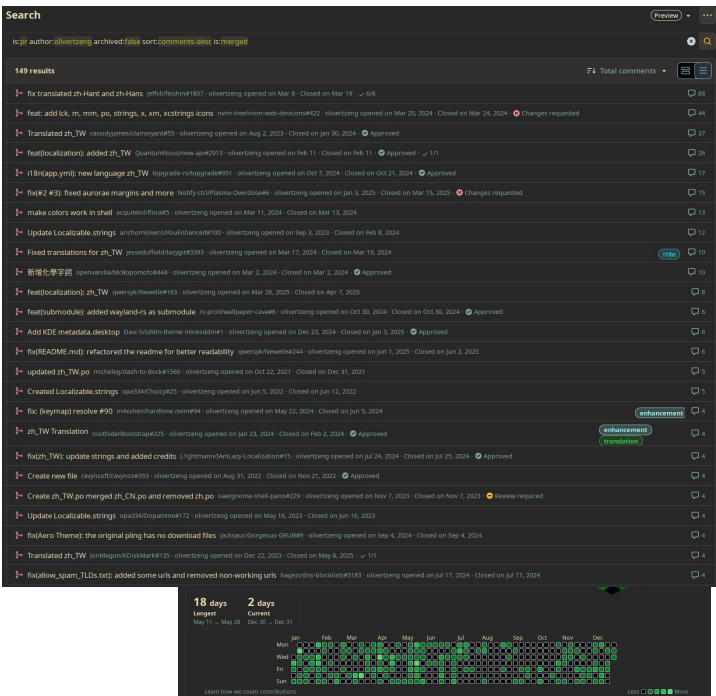
後期：效能優化與格式擴展

專案數量增加後，逐檔處理的速度成為瓶頸。架構上改用 fd 與 ripgrep 取代原本的 find 與 grep，大幅提升檔案搜尋與內容匹配的速度，並加入並行處理模式應對大型專案。同時擴展支援格式至 JSON value 的選擇性翻譯。

▼149個 merged PR，總共200

持續維護

工具的字典檔存放於我的 dotfiles 中，根據自己在各專案提交 PR 時遇到的新案例持續更新。每一次新的誤判都會回饋到字典中，讓工具的覆蓋率逐步提升。



四、成果

利用 **replace.sh** 處理簡體轉繁體的翻譯，搭配手動英文翻中文的工作流程，累計約 150 個 pull request 被上游專案合併，涵蓋專案包括 **topgrade-rs**、**yay**、**BleachBit** 等開源工具。工具本身開源於 **GitHub**，供其他貢獻者使用。

五、反思

從偷懶到系統化

從最初的簡單替換腳本到目前約 1800 行的工具，最大的收穫是學會把「想偷懶」變成一個真正能解決問題的系統。一開始只是覺得手動翻譯很麻煩，後來慢慢拆解出哪些能自動修、哪些不能動。

AI 輔助與人工把關的分工

開發過程中雖然部分依賴 AI 工具輔助產出程式碼，但功能的規劃、歧義詞的判斷與每一筆翻譯的品質把關始終是我自己的工作。

自動化思維的延伸

replace.sh 的經驗讓我養成了一個習慣：當我發現自己在不同專案反覆做同樣的事，就開始想辦法把它自動化。後續我開發 **yt.py** 自動化音樂庫管理，並以 **Docker** 部署十多項自架服務，皆是這套邏輯的延伸。這個從抱怨、拆解到實作的過程，是我高中階段最扎實的一課。

附錄

- **replace.sh** GitHub 連結：<https://github.com/olivertzeng/dotfiles/blob/main/replace.sh>
- **yt.py** GitHub 連結：<https://github.com/olivertzeng/dotfiles/blob/main/yt.py>

▲2025年於GitHub的活動

▼replace.sh —help 頁面

```
Advanced Chinese Converter & PNG Metadata Tool
Uses fd + rg for fast file discovery and content matching.

Usage: replace.sh [OPTIONS] [files...]

Conversion Options:
  -c          Enable controversial replacements
  -r          Reverse mode: TW -> CN (default: CN -> TW)
  -i          In-place mode: overwrite original files
              Also translates filenames (刪除.txt -> 刪除.txt)
  -R          Recursive: process directories recursively
              (auto-enabled when input is a directory)
  -u          Unify: force -TW/-CN suffix on output
  --new-api   Translate JSON "translation" values only (keys untouched)

PNG Operations:
  -j [MODE]   PNG extraction: o(original), c(n), t(w)
              cn/tw modes also translate filename
  -a          Amend mode: combine PNG + JSON into character card
              Usage: -a <file1> <file2> [-o output.png]

File Handling:
  -H          Include hidden files
  -g          Respect .gitignore (default: on)
  -b          Create backup (.bak)
  -o [files]  Specify output filenames
  -s, --stage Auto git-stage processed files

Execution:
  -n          Dry-run (no changes made)
  -p          Parallel processing (faster for many files)
  -f          Force continue on errors
  -q, --quiet Quiet mode (errors only)
  -l, --log FILE Write log to file
  --no-confirm Skip confirmation prompt for -i -R

Help:
  --help     Show this help

Examples:
  replace.sh -i file.txt           In-place convert (CN->TW)
  replace.sh -i -r file.txt       In-place convert (TW->CN)
  replace.sh ./zh-CN/             Copy to ./zh-TW/ with converted files
  replace.sh -i ./zh-CN/          In-place convert with dir rename
  replace.sh -i -R -p ./dir/       Parallel recursive conversion
  replace.sh -i -R -b -s ./dir/    With backup and auto-stage
  replace.sh -a image.png data.json Combine into character card
  replace.sh --new-api -i file.json Translate translation values only

Supported Extensions:
  txt json jsonl md po strings png PNG yaml yml xyaml js mjs html css scss

Blacklist:
  *.bak *.tmp .DS_Store .git LICENSE __pycache__ build dist node_modules
  ...

[END]
```